Zadanie 2 16-19 marca 2026 r.

kurs języka C++

figury na płaszczyźnie

Instytut Informatyki
Uniwersytetu Wrocławskiego Paweł Rzechonek

Prolog

Geometria euklidesowa to klasyczna odmiana geometrii opisana po raz pierwszy przez Euklidesa w dziele Elementy z IV wieku p.n.e. (zebrał on w tym dziele całą ówczesną wiedzę matematyczną znaną Grekom). W geometrii euklidesowej występują pojęcia pierwotne, przyjmowane bez definicji, do których należą punkt, prosta, płaszczyzna oraz należenie punktu do prostej i należenie prostej do płaszczyzny. Euklides przyjął, że punkt, prosta, płaszczyzna i przestrzeń mają wymiary kolejno: zero, jeden, dwa i trzy.

Przekształcenie płaszczyzny euklidesowej jest nazywane przekształceniem izometrycznym, gdy zachowuje odległości dowolnej pary punktów — innymi słowy, jeśli jakąś figurę przekształcimy za pomocą izometrii, to nie zmieni ona ani kształtu ani rozmiaru. W geometrii euklidesowej istnieją cztery zasadnicze przekształcenia płaszczyzny:

1. przesunięcie/translacja – przesunięcie wszystkich punktów płaszczyzny/figury o zadany wektor;
2. obrót – obrót wszystkich punktów płaszczyzny/figury wokół ustalonego punktu o zada-ny kąt;
3. symetria środkowa – odbicie wszystkich punktów płaszczyzny/figury względem ustalonego punktu;
4. symetria osiowa – odbicie wszystkich punktów płaszczyzny/figury względem ustalonej osi.

Dwie figury definiuje się jako przystające, jeżeli jedna z nich może być przekształcona w drugą za pomocą przesunięć, obrotów i symetrii. Przesunięcia, obroty i symetrie tworzą grupę przekształceń.

Zadanie

Zdefiniuj strukturę `punkt` kategorii POD (ang. Plain Old Data), która będzie zawierać dwa pola `x` i `y` typu `double` do pamiętania współrzędnych punktu na płaszczyźnie euklidesowej z kartezjańskim układem współrzędnych. Dalej zdefiniuj klasy `odcinek` i `trojkat`, które będą reprezentowały odpowiednio odcinek i trójkąt na płaszczyźnie euklidesowej. Klasa `odcinek` ma być zdefiniowana za pomocą dwóch różnych punktów będących końcami odcinka. Klasa `trojkat` ma być zdefiniowana za pomocą trzech niewspółliniowych punktów. Pamiętaj o hermetyzacji, aby ukryć stan odcinka i trójkąta.

W strukturze opisującej punkt zdefiniuj funkcje składowe do wykonywania przekształceń izometrycznych na tych obiektach geometrycznych (przesunięcia, obroty oraz symetrie środkowe i osiowe). Ponadto zdefiniuj funkcję globalną, która będzie wyznaczać odległość pomiędzy parą punktów.

W klasach opisujących odcinek i trójkąt zdefiniuj konstruktory (w tym konstruktor kopiujący), przypisanie kopiujące oraz funkcje składowe do wykonywania przekształceń izometrycznych.

W klasie `odcinek` zdefiniuj funkcję składową obliczającą długość odcinka oraz funkcję składową sprawdzającą, czy zadany punkt należy do odcinka. Dodatkowo zdefiniuj dwie funkcje globalne: jedna ma sprawdzać czy dwa odcinki są równoległe a druga czy są prostopadłe.

W klasie `trojkat` zdefiniuj funkcje składowe obliczające obwód trójkąta i pole trójkąta oraz funkcję składową sprawdzającą, czy zadany punkt leży wewnątrz trójkąta. Dodatkowo zdefiniuj dwie funkcje globalne: jedna ma sprawdzać czy dwa trójkąty są rozłączne a druga czy jeden trójkąt zawiera się w drugim.

Do wykonania translacji będziesz potrzebować wektora — zdefiniuj go w postaci odrębnej struktury kategorii POD. W strukturze tej umieść stałe pola `dx` i `dy` typu `double`.

Do wykonania symetrii osiowej będziesz potrzebować prostej — także zdefiniuj ją w postaci odrębnej struktury kategorii POD (w postaci równania ogólnego Ax + By + C = 0, przy czym A i B nie mogą być jednocześnie równe 0). W strukturze tej umieść stałe pola `a`, `b` i `c` typu `double`.

Na koniec napisz program rzetelnie testujący działanie obiektów tych klas. W programie testującym uruchom wszystkie funkcje globalne oraz wszystkie publiczne funkcje składowe. Rezultaty wywołanych funkcji zaprezentuj na standardowym wyjściu. Wszystkie obiekty w tym programie powinny być utworzone na stosie.

Podziel program na pliki nagłówkowe i źródłowe. Funkcję `main()` z programem testującym umieść w osobnym pliku źródłowym.

Uwaga

Zawsze, gdy napotykamy w programie jakieś błędy, niejednoznaczności czy sprzeczności należy to sygnalizować na pomocą wyjątków. Sytuacje wyjątkowe zgłaszamy instrukcją `throw`. Na przykład w konstruktorze klasy odcinek należy zgłosić wyjątek, gdy oba końce odcinka będą miały takie same współrzędne – wtedy zgłoszony wyjątek powinien być typu `invalid_argument` (deklaracje tego i innych klas wyjątków standardowych znajdują się w pliku nagłówkowym `<stdexcept>`).

Żeby przekonać i siebie i mnie, że zdefiniowane struktury są kategorii POD, użyj w programie testowym trejta typowego `is_pod<>`, na przykład w taki sposób:

```
if (is_pod<punkt>::value) cout << "punkt jest POD" << endl;
else cout << "punkt nie jest POD" << endl;
```

Ważne elementy programu

- Podział programu na pliki nagłówkowe i źródłowe (definiowanie klas w pliku nagłówkowym i funkcji składowych w pliku źródłowym).
- Ukrywanie stanu w definicjach klas i upublicznienie funkcjonalności (hermetyzacja).
- Definicja konstruktorów kopiujących i operatorów przypisania (również za pomocą frazy `=default`).
- Wykorzystanie struktury `punkt` przy budowie odcinka i trójkąta (kompozycja).
- Inicjalizacja wybranych pól składowych w tworzonym obiekcie poprzez listę inicjalizacyjną w konstruktorze.
- Inicjalizacja obiektów kategorii POD poprzez listę wartości.
- Implementacja algorytmów geometrii analitycznej (przekształcenia izomorficzne w układzie kartezjańskim).
- Zgłaszanie wyjątków, powodowanych przekazywaniem błędnych parametrów.
- Program powinien zostać napisany w jakimś przyjaznym IDE (na przykład *CLion* firmy *JetBrains*).
- Program rzetelnie testujący wszystkie zdefiniowane w programie klasy i struktury wraz z przekształceniami izomorficznymi, umieszczony w funkcji `main()`.